Server Error Alerting Implementation Plan

> ***For agentic workers:*** *REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.*

Goal: Send email alerts to 3 admin addresses whenever the SkillSphere backend crashes or a critical controller operation fails.

Architecture: A shared `alertService.js` provides a throttled `sendAlert()` function (reuses existing Nodemailer transporter config). `server.js` registers `uncaughtException` / `unhandledRejection` process hooks for JS-level crashes. A bash wrapper `start.sh` replaces `node server.js` as the PM2 entry point so that process-level exits (OOM, SIGKILL) also trigger a standalone `pm2-crash-notify.js` script. Key controller catch blocks call `sendAlert()` for operational failures.

Tech Stack: Node.js, Nodemailer (already installed), PM2 ecosystem config, bash

Spec: `docs/server-error-alerting-approaches.md`

Global Constraints

- No new npm dependencies — Nodemailer is already installed
- SMTP credentials stay in `.env`: `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM_EMAIL`
- Alert recipients stored in `.env` as `ALERT_EMAILS=a@x.com,b@x.com,c@x.com`
- Throttle: 1 alert per unique error message per 15 minutes (in-memory Map)
- Crash hooks (`uncaughtException`, `unhandledRejection`, PM2 exit) are **never** throttled
- `alertService.js` must never throw — if email fails, log and swallow silently
- Backend root: `/home/skillsph/skill-sphere-Cpanel/backend/` on cPanel

Review Focus

- **SMTP down when crash occurs** — `pm2-crash-notify.js` must not hang the process exit; wrap `sendMail` in a `Promise.race` with a 5 s timeout.
- **`unhandledRejection` with non-Error reason** — `reason` may be a string or object, not always an `Error`; coerce to `Error` before passing to `sendAlert`.
- **Throttle survives PM2 restart** — in-memory Map resets on restart; the same error fires again after restart. Acceptable (documented as a known behaviour, not a bug).
- **`ALERT_EMAILS` not set** — `sendAlert` must exit silently (no crash) when the env var is empty or missing.
- **`start.sh` exit code forwarding** — the bash wrapper must forward Node's exit code to PM2 so PM2 still marks the process as errored and triggers its own restart logic.

File Map

Action	File	Responsibility
CREATE	`backend/services/alertService.js`	Throttled email alert sender
MODIFY	`backend/server.js`	Register crash hooks, hook global error handler
MODIFY	`backend/controllers/authController.js`	`sendLoginOTP`, `forgotPassword`, `login` catch blocks

MODIFY	`backend/controllers/paymentController.js`	Main catch block
MODIFY	`backend/controllers/enrollmentController.js`	Main catch block
MODIFY	`backend/controllers/feedbackController.js`	Main catch block
CREATE	`backend/pm2-crash-notify.js`	Standalone email sender for process-exit scenario
CREATE	`backend/start.sh`	Bash wrapper — runs Node, calls notify on non-zero exit
CREATE	`backend/ecosystem.config.js`	PM2 config pointing to `start.sh`
MODIFY	`backend/.env`	Add `ALERT_EMAILS`

Task 1: Create `alertService.js`

Files:

- Create: `backend/services/alertService.js`

Interfaces:

- Produces:
  - `sendAlert({ title: string, error: Error, context?: string }): Promise<void>` — throttled, never throws
  - `sendCrashAlert({ title: string, error: Error, context?: string }): Promise<void>` — unthrottled, never throws

- [ ] **Step 1: Create `backend/services/alertService.js`**

```
const nodemailer = require('nodemailer');
const throttleMap = new Map(); // key: string -> lastSentMs: number
const THROTTLE_MS = 15 * 60 * 1000;

function buildTransporter() { /* same config as emailService.js */ }
function getRecipients() { /* parse process.ALERT_EMAILS */ }
function buildBody(title, error, context) { /* plain-text body */ }
async function _send(title, error, context) { /* sendMail, 5 s timeout */ }
async function sendAlert({ title, error, context = '' }) { /* throttle check then _send */ }
async function sendCrashAlert({ title, error, context = '' }) { /* _send directly, no throttle */ }
module.exports = { sendAlert, sendCrashAlert };
```

  - `buildTransporter()` must mirror the config in `emailService.js` exactly: `host`, `port`, `secure`, `auth`, `tls.rejectUnauthorized: false`
  - `getRecipients()` returns `[]` (not throws) when `ALERT_EMAILS` is unset
  - `_send` wraps `transporter.sendMail` in `Promise.race([sendMail(...), timeout(5000)])` — on timeout or send failure, `console.error` and return (do not throw)
  - Throttle key: `error.message?.slice(0, 80) || title`
  - Email subject: `[SkillSphere ALERT] ${title}`

- Email body (plain text):

```
Environment : <NODE_ENV>
Time        : <ISO timestamp>
Context     : <context>

Error       : <error.message>

Stack:
<error.stack>
```

- [ ] **Step 2: Verify — manually run the file with a test call**

```
cd backend
node -e "require('./services/alertService').sendAlert({ title: 'Test', error: new Error('test'), context: 'manual test' }).then(() => console.log('done'))"
```

Expected: no crash, email arrives in your inbox (or `getRecipients returned []` logged if `ALERT_EMAILS` not yet set)

- [ ] **Step 3: Commit**

```
git add backend/services/alertService.js
git commit -m "feat: add alertService with throttled email alerts"
```

Task 2: Crash Hooks in `server.js`

Files:

- Modify: `backend/server.js` — top of file + global error handler (lines 223–228)

Interfaces:

- Consumes: `sendCrashAlert`, `sendAlert` from `./services/alertService`

- [ ] **Step 1: Add crash hooks at the very top of `server.js`** — immediately after `require('dotenv').config(...)`, before any other require

```
const { sendCrashAlert, sendAlert } = require('./services/alertService');

process.on('uncaughtException', async (error) => {
  await sendCrashAlert({ title: 'CRASH — uncaughtException', error, context: 'Process will exit' });
  process.exit(1);
});

process.on('unhandledRejection', async (reason) => {
  const error = reason instanceof Error ? reason : new Error(String(reason));
  await sendCrashAlert({ title: 'unhandledRejection', error, context: 'Unhandled
```

```
promise rejection' });
});
```

- [ ] **Step 2: Hook the existing global error handler** at line ~223

  Change:

```
app.use((err, req, res, next) => {
  console.error('Error:', err);
  res.status(err.status || 500).json({ error: err.message || 'Internal server error'
});
});
```

  To:

```
app.use((err, req, res, next) => {
  console.error('Error:', err);
  sendAlert({ title: `Express error — ${err.message?.slice(0, 60)}`, error: err,
context: 'Global Express error handler' });
  res.status(err.status || 500).json({ error: err.message || 'Internal server error'
});
});
```

- [ ] **Step 3: Verify — start server and confirm no startup errors**

```
node server.js
```

  Expected: server starts normally, no new errors in output

- [ ] **Step 4: Commit**

```
git add backend/server.js
git commit -m "feat: add uncaughtException and unhandledRejection crash alert hooks"
```

Task 3: Controller Catch Blocks

Files:

- Modify: `backend/controllers/authController.js`
- Modify: `backend/controllers/paymentController.js`
- Modify: `backend/controllers/enrollmentController.js`
- Modify: `backend/controllers/feedbackController.js`

Interfaces:

- Consumes: `sendAlert` from `../services/alertService`

Pattern for every file — add at top:

```
const { sendAlert } = require('../services/alertService');
```

Then in each relevant catch block, add **one line before the existing** `console.error` or `res.status(500)`:

```
sendAlert({ title: '<FunctionName> failed', error, context: '<METHOD /route/path>' });
```

- [ ] **Step 1: `authController.js` — add import and three catch blocks**

  Target functions and their context strings:

Function	Context
`exports.sendLoginOTP` (line ~488 catch)	`'POST /auth/login-otp'`
`exports.forgotPassword` (line ~604 catch)	`'POST /auth/forgot-password'`
`exports.login` (top-level catch)	`'POST /auth/login'`

- [ ] **Step 2: `paymentController.js` — add import and catch blocks**

  Add `sendAlert` to any catch block that calls `res.status(500)`. Context string format: `'POST /payments/<action>'`

- [ ] **Step 3: `enrollmentController.js` — add import and catch blocks**

  Target: any catch block that calls `res.status(500)`. Context string format: `'POST /enrollments/<action>'`

- [ ] **Step 4: `feedbackController.js` — add import and catch blocks**

  Target: `getFeedbackByCourse` catch block. Context: `'GET /feedback/by-course'`

- [ ] **Step 5: Verify — trigger a known error and check email**

  In `authController.js` `sendLoginOTP`, temporarily force an error before user.update to confirm an alert email arrives. Remove the forced error after confirming.

- [ ] **Step 6: Commit**

```
git add backend/controllers/authController.js
backend/controllers/paymentController.js backend/controllers/enrollmentController.js
backend/controllers/feedbackController.js
git commit -m "feat: add sendAlert to critical controller catch blocks"
```

Task 4: PM2 Crash Notify Script + Bash Wrapper

Files:

- Create: `backend/pm2-crash-notify.js`
- Create: `backend/start.sh`

Interfaces:

- `pm2-crash-notify.js` is a standalone script — no imports from alertService (self-contained, same SMTP logic inline so it works even if alertService has a bug)
- `start.sh` calls `node server.js`, captures exit code, runs `node pm2-crash-notify.js $EXIT_CODE` on non-zero exit, then `exit $EXIT_CODE`
- [ ] **Step 1: Create `backend/pm2-crash-notify.js`**

  Self-contained file. Inlines the Nodemailer transporter (same SMTP env vars). No throttle — this only runs when the process exits.

```
require('dotenv').config({ path: require('path').resolve(__dirname, '.env') });
const nodemailer = require('nodemailer');

const exitCode = process.argv[2] || 'unknown';

async function notify() {
  const recipients = (process.env.ALERT_EMAILS || '').split(',').map(s =>
s.trim()).filter(Boolean);
  if (!recipients.length) { console.log('ALERT_EMAILS not set — skipping crash
notify'); return; }

  const transporter = nodemailer.createTransport({
    host: process.env.SMTP_HOST,
    port: parseInt(process.env.SMTP_PORT || '587', 10),
    secure: parseInt(process.env.SMTP_PORT || '587', 10) === 465,
    auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS },
    tls: { rejectUnauthorized: false },
  });

  await transporter.sendMail({
    from: process.env.SMTP_FROM_EMAIL,
    to: recipients.join(', '),
    subject: `[SkillSphere ALERT] Process exited — code ${exitCode}`,
    text: `SkillSphere server process exited with code ${exitCode} at ${new
Date().toISOString()}.\n\nThis was caught by the PM2 bash wrapper (start.sh) — likely
OOM or SIGKILL.\n\nCheck PM2 logs: pm2 logs skillsphere --lines 50`,
  });

  console.log('PM2 crash alert sent');
}

notify().catch(e => console.error('pm2-crash-notify failed:', e.message));
```

- [ ] **Step 2: Create `backend/start.sh`**

```
#!/bin/bash
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
node "$SCRIPT_DIR/server.js"
EXIT_CODE=$?
```

```
if [ $EXIT_CODE -ne 0 ]; then
  node "$SCRIPT_DIR/pm2-crash-notify.js" $EXIT_CODE
fi
exit $EXIT_CODE
```

- [ ] **Step 3: Make `start.sh` executable**

```
chmod +x backend/start.sh
```

- [ ] **Step 4: Verify `pm2-crash-notify.js` works standalone**

```
cd backend
node pm2-crash-notify.js 137
```

Expected: alert email arrives with subject `[SkillSphere ALERT] Process exited — code 137`

- [ ] **Step 5: Commit**

```
git add backend/pm2-crash-notify.js backend/start.sh
git commit -m "feat: add PM2 crash notify script and bash wrapper"
```

Task 5: `ecosystem.config.js` + `.env` + cPanel Deploy

Files:

- Create: `backend/ecosystem.config.js`
- Modify: `backend/.env` (cPanel only — not committed)

Interfaces:

- `ecosystem.config.js` points PM2 to `start.sh` instead of `server.js`

- [ ] **Step 1: Create `backend/ecosystem.config.js`**

```
module.exports = {
  apps: [{
    name: 'skillsphere',
    script: './start.sh',
    interpreter: 'bash',
    cwd: '/home/skillsph/skill-sphere-Cpanel/backend',
    env_production: {
      NODE_ENV: 'production',
    },
    max_restarts: 10,
    restart_delay: 3000,
    log_date_format: 'YYYY-MM-DD HH:mm:ss',
    error_file: '/home/skillsph/.pm2/logs/skillsphere-error.log',
    out_file: '/home/skillsph/.pm2/logs/skillsphere-out.log',
```

```
  }]
};
```

- [ ] **Step 2: Add `ALERT_EMAILS` to `backend/.env` on cPanel**

```
ALERT_EMAILS=youremail@gmail.com,secondemail@gmail.com,thirdemail@gmail.com
```

This file is not committed. Update it directly on the server via cPanel File Manager.

- [ ] **Step 3: On cPanel — deploy files and switch PM2 to ecosystem config**

Upload to server:

- `backend/services/alertService.js`
- `backend/server.js`
- `backend/controllers/authController.js`
- `backend/controllers/paymentController.js`
- `backend/controllers/enrollmentController.js`
- `backend/controllers/feedbackController.js`
- `backend/pm2-crash-notify.js`
- `backend/start.sh`
- `backend/ecosystem.config.js`

Then on the server:

```
chmod +x /home/skillsph/skill-sphere-Cpanel/backend/start.sh
pm2 delete skillsphere
pm2 start /home/skillsph/skill-sphere-Cpanel/backend/ecosystem.config.js --env
production
pm2 save
```

- [ ] **Step 4: Verify full stack**

Check logs:

```
pm2 logs skillsphere --lines 30
```

Expected: server starts normally, no errors

Trigger a test alert:

```
node /home/skillsph/skill-sphere-Cpanel/backend/pm2-crash-notify.js 999
```

Expected: alert email arrives within 30 seconds

- [ ] **Step 5: Commit `ecosystem.config.js`**

```
git add backend/ecosystem.config.js backend/start.sh
git commit -m "feat: add PM2 ecosystem config with bash wrapper entry point"
```

Summary

After all tasks complete:

Scenario	Covered By
JS crash (`throw` without catch)	`uncaughtException` hook in `server.js`
Unhandled promise rejection	`unhandledRejection` hook in `server.js`
OTP save fails (DB column too small, etc.)	`sendAlert` in `authController` catch
Payment failure	`sendAlert` in `paymentController` catch
Enrollment failure	`sendAlert` in `enrollmentController` catch
Express middleware error	Global error handler hook in `server.js`
OOM kill / SIGKILL (PM2 restarts)	`start.sh` + `pm2-crash-notify.js`
PM2 restart loop	`max_restarts: 10` + `pm2-crash-notify.js` fires each time